

Clinical Memory AI

Interview Preparation

September 2026

How to use this

This is the **performance layer**: what to say, in what order, with which numbers. It assumes you understand the system — if you do not yet, read `CLINICAL_MEMORY_AI_PROJECT_GUIDE.md` first, then come back here.

Study it in this order:

1. **The numbers** (page 2) — memorise the table. Nothing costs you more credibility than “around ninety percent, I think”.
2. **The four pitches** (page 3) — rehearse the 60-second one out loud until it is automatic. Every interview starts there.
3. **The five stories** (page 5) — these are your answers to “tell me about a bug”, “a hard technical decision”, “something you got wrong”. Each is a real incident with a real fix.
4. **The questions** (page 8) — twenty-five, ordered by how likely you are to be asked. The remaining twenty-five are in the project guide.
5. **The traps** (page 12) — read this last, the night before.

One principle underneath all of it: **every claim you make should have a number or a test behind it, and you should be able to say which**. The project was built that way on purpose. Answer that way too.

The numbers

Memorise the left column. Know where each one comes from.

Metric	Value	Where it comes from
Tests	426	290 backend, 98 database, 38 frontend
Risk model ROC-AUC	0.90	held-out synthetic, n=1500
Risk model PR-AUC	0.79	baseline (positive rate) is 0.20
Recall / sensitivity	0.82	at the 0.11 operating threshold
Precision	0.66	threshold chosen for precision ≥ 0.60
Specificity	0.90	
Calibration error (ECE)	0.10 $\rightarrow$ 0.01	before $\rightarrow$ after isotonic calibration
Citation verification	precision 1.00, recall 1.00	labelled synthetic set, gated in CI
Red-flag sensitivity	12/12	end-to-end, model + deterministic criteria
False-alarm rate	1/12 (8%)	on controls that include near-misses
Features in the model	46	all named, all binary or bounded
Inference parity vs sklearn	4.8e-05	max delta over 1500 rows

Three things to say about the numbers, unprompted:

- “The baseline PR-AUC is 0.20, so 0.79 is the meaningful comparison.” Quoting PR-AUC without its baseline is the same mistake as quoting accuracy.
- “I don’t lead with accuracy — predicting *never escalate* scores 80% here.”
- “The training data is synthetic, so these establish that the pipeline is correct, not that the model works on real patients.”

Say the third one **before** they ask. It is the single most important sentence in your interview, and volunteering it converts a weakness into evidence of judgement.

The four pitches

30 seconds

Clinical Memory AI turns a spoken doctor–patient consultation into a structured clinical note, and carries the patient’s history across visits as measurable signal rather than as a list. Next.js frontend, FastAPI backend, PostgreSQL on Supabase — where multi-tenant isolation is enforced by Row-Level Security rather than by application code. Everything the AI produces is a draft; the database refuses to finalise a note without an explicit physician attestation.

60 seconds

(the above, then:)

The interesting engineering is in not trusting the AI. Every field the model auto-fills carries the verbatim words from the transcript that produced it, and the server verifies each quote actually appears there before showing it — precision 1.0 on a labelled set, and gated at 1.0 in CI, because showing a fabricated quote as verified evidence converts a physician’s scepticism into misplaced trust.

There’s also an interpretable ML layer: a calibrated logistic regression over 46 named clinical features that answers one question — is this case worth a second look? ROC-AUC 0.90, recall 0.82, calibration error down from 0.10 to 0.01. It’s trained on synthetic data, so I present it as unvalidated — that’s stated in the API response and printed in the UI next to the score.

And the longitudinal memory does real statistics: Mann–Kendall trend tests and robust step detection, so it can say “this patient’s blood pressure is genuinely rising, p equals 0.03” rather than just listing old readings.

2 minutes

Add, in this order:

The problem. A doctor has ten minutes to listen, examine, recall history, decide on tests, prescribe — and type. The typing loses. History gets lost between visits, and there’s usually no record of where any given piece of information came from.

The constraint that shaped everything. A clinical note is a legal document and AI output is unreliable. So the system is built so AI output is always a *proposal*, and only a physician’s signature makes it a *fact*. That’s why the rules live in Postgres rather than in the API: signing is one function, in one transaction, that requires attestation, requires the doctor role, refuses to re-sign a signed visit, and rejects a stale version. Those hold even if my API code is wrong.

What I’d highlight. Isolation is RLS — no query in my codebase filters by clinic, and 31 database tests actively try to cross the boundary. The audit log is append-only by trigger and hash-chained, with a `verify_audit_chain()` function so tamper-evidence is a query, not a claim. Provider failover is real and

cross-vendor, with the fallback rate exported as a metric.

What I'd be honest about. The differential-diagnosis engine is an external API — I built the integration, not the clinical brain. The risk model is trained on synthetic data. It isn't deployed. Rate limiting is per-process.

5 minutes (technical)

Same opening, then walk these six, roughly 45 seconds each:

1. **Isolation.** RLS, `current_clinic_id()`, `USING` vs `WITH CHECK`, why not application filters. Land the missing-UPDATE-policy bug (story 1).
2. **Transactional finalize.** Five REST calls → one function. Attestation, role, immutability, optimistic locking, all in the database. Two connections racing in a test.
3. **AI reliability.** Ordered chains, cross-vendor, transient vs non-transient, structure-only JSON repair, and the rule that a 200 carrying prose is a failure.
4. **Citation verification.** The exact / near / unsupported ladder, and why the content-word requirement is the safety property. Numbers.
5. **The ML.** Why logistic regression over the gradient-boosted tree I actually trained. Calibration and why class weighting makes it necessary. The threshold choice and alarm fatigue. The parity gate (story 2).
6. **Evaluation as regression testing,** and the two real bugs it found.

Closing line, have it ready:

The thing I'd most want to be judged on isn't any single feature — it's that every claim in the README has a test or a number behind it, and the ones that didn't have been removed.

The five stories

Each is a real incident. Tell them as: **what looked wrong** → **what was actually wrong** → **what I changed** → **how I stopped it recurring**. That last clause is what separates a debugging anecdote from an engineering story.

Story 1 — The silent 204

Use for: “tell me about a bug”, “a time you found something serious”, *RLS* questions.

`soap_notes` had Row-Level Security enabled with SELECT, INSERT and DELETE policies — but no UPDATE policy. PostgREST returns **HTTP 204 for an update that matches zero rows**, so finalising a draft reported success and wrote nothing. Clinical data loss that was indistinguishable from success, in the exact operation where losing data matters most.

It only worked in production because the live database schema had drifted from the committed migrations.

I found it by building the database from the committed migrations in CI — which had never been done. The fix was one policy. The *real* fix was a structural test asserting that every writable clinical table has SELECT, INSERT and UPDATE policies, so the class of bug cannot recur.

The line that lands: “The dangerous part wasn’t the missing policy. It was that a failed write and a successful write returned the same status code.”

Story 2 — The calibrator that lied

Use for: “a hard technical decision”, *ML* questions, “how do you know your code is right”.

The risk model is trained with scikit-learn but served by hand-written pure Python, so the API has no ML dependency and the artefact is JSON rather than a pickle — loading a pickle is arbitrary code execution, which isn’t a property a clinical system should ship with.

The price of that choice is that the two implementations can disagree. So the training script runs my inference over the entire test set and **refuses to write the artefact** if it diverges from scikit-learn by more than $1e-4$.

It caught a real one. `CalibratedClassifierCV` fits its calibrator on `decision_function()` — log-odds — not `predict_proba()`, whenever the base estimator exposes one. I was feeding it the sigmoid output. The probabilities looked entirely plausible: an ankle sprain scored 0.23 against a threshold of 0.10. Every benign case flagged. Nothing crashed.

The line that lands: “I wrote the check before I needed it, which is the only reason I found it — the wrong numbers looked completely reasonable.”

Story 3 — The evaluation that scored dead code

Use for: “something you got wrong”, testing philosophy, “how do you measure quality”.

The project had a red-flag evaluation harness reporting a recall figure. It scored `kb_ground_red_flags()` — a knowledge-base matcher that had been removed from the request path months earlier because it surfaced conditions unrelated to the presentation (liver cancer for an ankle sprain). It also required a live database, so it never ran in CI.

So it produced a number that looked like evidence for a code path no consultation ever touched. That is worse than having no evaluation.

I rewrote it to score the path that actually ships, made it run with no database and no API key, and wired it into CI as a hard gate: 12/12 sensitivity, 1/12 false alarms.

The line that lands: “A benchmark that isn’t in CI drifts, and a drifted benchmark is worse than none — it gives you false confidence with a number attached.”

Story 4 — The chain that broke on its own timestamps

Use for: databases, concurrency, “attention to detail”.

The audit log is hash-chained: each row carries a SHA-256 over its contents plus its predecessor’s hash. The trigger picked that predecessor with `order by at desc, id desc`.

`at` defaults to `now()`, which in PostgreSQL is the **transaction** start time — identical for every row written in one transaction. So the tie broke on `id`, a random UUID, and rows chained in UUID order rather than insertion order. And `finalize_visit()` writes its audit row *inside* the clinical transaction, so this was reachable in completely normal use.

Fixed by chaining on a monotonic sequence. I also added `verify_audit_chain()`, so “tamper-evident” is something you can run rather than something I assert — and a test that disables the trigger as the table owner, rewrites a row, and asserts the verifier catches it.

The line that lands: “`now()` is transaction time, not statement time. That one word is the whole bug.”

Story 5 — Ten seconds of blocked event loop

Use for: performance, async, code review, “what did a reviewer catch”.

Citation verification slides a window across the transcript looking for a near match. It was scanning every offset at three window widths. At the 60,000-character transcript cap that measured **0.56 seconds of CPU for a single quote** — and a full extraction verifies up to seventeen. Roughly ten seconds of blocked event loop inside an `async` handler, stalling every other request in the process, including other doctors’ live consultation polls.

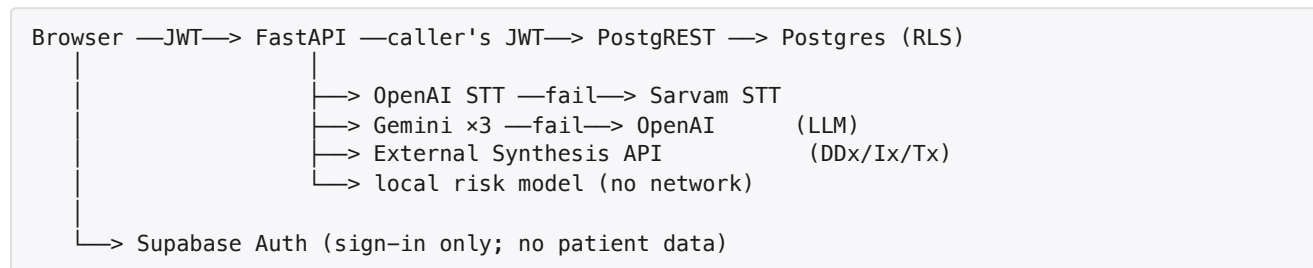
The fix is algorithmic rather than threading it: a near match must contain every content word of the quote, so it must overlap a position where the quote’s **rarest** content word occurs. Anchoring the search on those positions took it to 9.5 milliseconds — sixty times faster — with every verdict unchanged, plus a hard cap on anchor count so a pathological transcript can’t reintroduce it.

The line that lands: “It wasn’t slow code, it was a blocked event loop. `async def` doesn’t make CPU work concurrent.”

Three diagrams to be able to draw

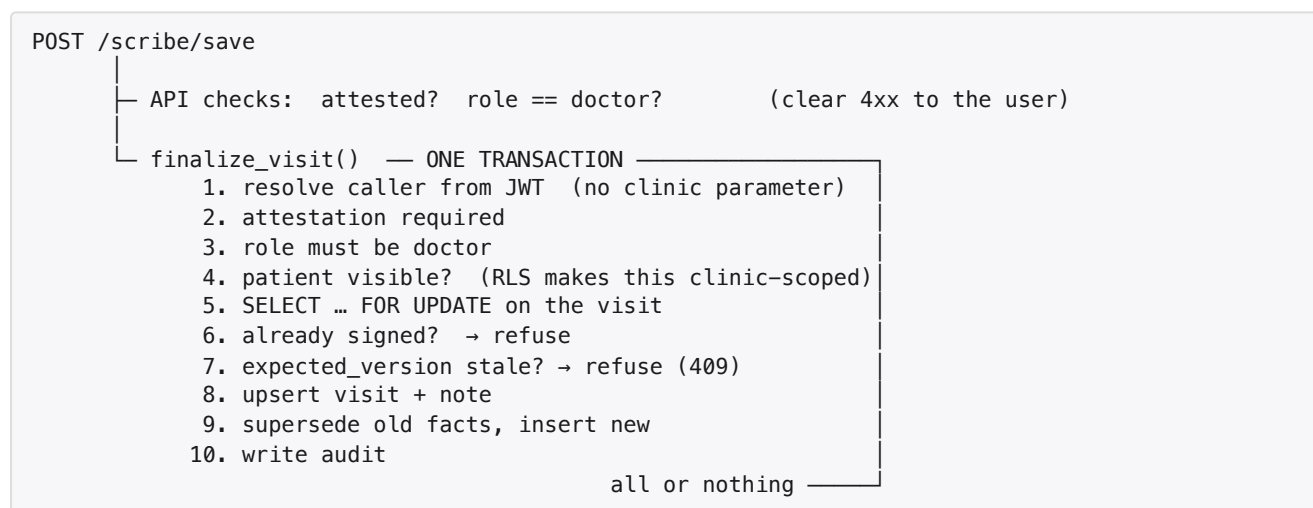
Practise these on paper. Being able to draw the system while talking is worth more than any single answer.

1. The request path



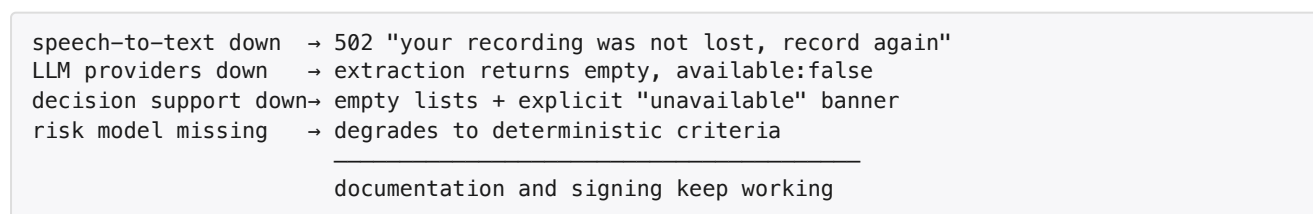
The point to make while drawing it: the backend forwards the *caller's own token* to the database, so RLS is evaluated against their identity. That one arrow is the entire multi-tenancy design.

2. The save path



The point: steps 2, 3, 6 and 7 are in the database, not the API. They hold even if the API is wrong or bypassed entirely.

3. The degradation ladder



The point: each rung is a test. And the banner matters as much as the fallback — a doctor reading an empty differential as reassurance is the dangerous failure.

Twenty-five questions

Ordered by likelihood. The remaining twenty-five, covering Python, TypeScript, speech, RAG and system design, are in the project guide.

Tier 1 — you will be asked these

- 1. Walk me through the project.** Use the 2-minute pitch. Stop talking at two minutes.
- 2. Explain Row-Level Security to someone who hasn't used it.** A `WHERE` clause the database enforces on every query, no matter who wrote the query. Every clinical table restricts rows to `clinic_id = current_clinic_id()`, which resolves the caller's clinic from their JWT. The backend forwards the user's own token to PostgREST, so the database evaluates the rule against their identity. → *Follow-up: "why not filter in application code?"* → *Because that fails the day someone forgets one clause on one query. Here, no query filters by clinic at all.*
- 3. What's the difference between `USING` and `WITH CHECK`?** `USING` decides which rows you can see and target. `WITH CHECK` validates the rows you're **writing**. Without `WITH CHECK` on an `UPDATE`, you could take a row you legitimately own and set its `clinic_id` to another clinic — walking it out of the tenant. There's a test that tries exactly that. *This question separates people who wrote policies from people who copied them.*
- 4. Tell me about a bug you found.** Story 1 (the silent 204).
- 5. How do you stop the model hallucinating clinical facts?** Four things. The prompt forbids invention and tells the model evidence will be checked. Every auto-filled field must carry a verbatim quote. The server verifies each quote against the transcript and drops what it can't find. And nothing becomes a confirmed clinical fact until a physician attests — drafts write **zero** facts. → *Follow-up: "can it still hallucinate the note prose?"* → *Yes. Verification covers evidence, not prose. That's why the physician reviews and why the raw transcript is shown next to the note.*
- 6. Your data is synthetic. Isn't the evaluation meaningless?** *Want this question.* It's meaningless as a clinical claim, and I say so first — in the model card, the API response and the UI. What it establishes is that the pipeline is correct: features extract as intended, the model recovers a signal it can't see directly, calibration works, and the serving path matches the training path. It isn't circular, because the labels come from a deliberately non-linear rule — a threshold on the *count* of deranged vital systems, an age × severity interaction, a comorbidity multiplier that only applies to cardiorespiratory presentations, plus 4% label noise. A logistic regression can't recover that exactly. If I'd generated labels linearly, the metrics would measure nothing. → *Follow-up: "what would real validation need?"* → *A labelled retrospective cohort with recorded outcomes, ethics approval, and a prospective silent-mode evaluation before it influenced anyone's care.*
- 7. Why logistic regression and not something stronger?** I trained a gradient-boosted tree as a reference. It scored marginally better — ROC-AUC 0.909 versus 0.902. I didn't deploy it, because a physician has to be able to read why a case was raised and disagree with it. In a linear model the contribution of a feature is coefficient × value, so the explanation falls out of the arithmetic with no separate explainer that could

disagree with the model. Two points of AUC on synthetic data doesn't buy that. → *Follow-up: "what about SHAP?"* → *SHAP approximates the model. For a clinical prompt I'd rather the explanation be* the model — and I'd have to explain a Shapley value to a clinician.**

8. Explain precision, recall and F1 in this system's terms. Precision: of the cases we flagged, how many warranted a second look — 0.66. Recall: of the cases that warranted one, how many we caught — 0.82. F1 is their harmonic mean, 0.73. I report PR-AUC as the headline for the imbalanced view, because with a 20% positive rate the baseline PR-AUC is 0.20 and mine is 0.79 — ROC-AUC flatters imbalanced problems. → *Follow-up: "why not accuracy?"* → *"Never escalate" scores 80%. Accuracy is the metric that hides the failure.*

9. What's the most dangerous failure mode of this system? Silent wrongness. Not a crash — a crash is visible. The dangerous case is a plausible-looking note with a fabricated detail that a rushed physician signs. That's why citation verification exists, why the transcript is shown next to the note, and why the highest-priority CI gate is citation precision exactly 1.00. Second is the inverse: a doctor reading an empty decision-support panel as reassurance. Which is why "unavailable" is shown explicitly rather than as emptiness. *Most candidates name only the first. Naming both is the answer.*

10. What would you do with another week? In order: containerise and actually deploy it — the Cloudflare path is configured and unverified, and that's the biggest gap between "works on my machine" and "works". Then Redis for rate limiting and the spend cap, so they're correct across instances. Then a real end-to-end test against a deployed instance. Then code the diagnoses — differentials carry ICD-10 from the external service but what gets saved is free text, which weakens everything analytical downstream. What I *wouldn't* do is add features. The gap isn't capability.

Tier 2 — likely, especially with a senior engineer

11. How do two doctors editing the same consultation get handled? Optimistic locking. `GET /visits/{id}` returns `version`; the client sends it back; `finalize_visit()` does `SELECT ... FOR UPDATE` then compares. The second writer blocks on the row lock, then fails with `serialization_failure`, which maps to 409 and "reload and try again". Tested with two real connections racing on a committed draft. → *"Why optimistic rather than pessimistic?"* → *A consultation lasts ten minutes. Holding a lock that long blocks legitimate work and breaks if the browser closes. Conflicts are rare; detecting them is enough.*

12. Describe your provider failover. An ordered `(provider, model)` chain: all Gemini models, then OpenAI. Models within a vendor first, because model overload is the common failure; then a different vendor, because a fallback that shares an outage with its primary isn't a fallback. Transient errors advance. A 401 or 400 skips the rest of *that vendor's* models — a bad key won't get better — but still falls through to the next vendor. Fallbacks are counted as a metric. *The 401-skips-the-vendor rule is the detail that shows thought.*

13. How do you handle an LLM returning malformed JSON? Ask for JSON mode. Repair the structure if truncated — close what was left open, drop a dangling key. If it still won't parse, treat it as a **failure** and fall over, because a 200 carrying prose used to produce an empty note with no error anywhere. The rule is: repair structure, never invent content. A truncated field is dropped, not guessed. → *"How do you know the repair is safe?"* → *A property test truncates a real note at every offset and asserts the result is always a subset of the original with unchanged values.*

14. Why Mann–Kendall instead of linear regression? Three to twelve points, irregularly spaced, with occasional transcription errors. Least squares assumes normal residuals and is rotated by a single outlier — a mistyped 210 for a heart rate would produce a “significant trend”. Mann–Kendall uses only the sign of each pairwise comparison, so it’s robust to that and assumes no distribution. There’s a test with exactly that outlier asserting no trend is reported. → “What’s the cost?” → *Less power than least squares when the data really is well-behaved. For $n=5$ with a possible typo, robustness is worth more.*

15. Why do you have both a trend test and a step detector? They catch complementary patterns, and the second is the clinically interesting one. A patient at 72, 75, 71, 74, 73, then 120 has **no** monotonic trend — Mann–Kendall correctly says stable — but that jump is exactly what a doctor needs to see. The step detector uses median absolute deviation rather than standard deviation, because with five points one bad reading inflates the SD enough to hide the very jump you’re looking for. *Use the worked example. It makes the point instantly.*

16. What is calibration and why did you need it? A calibrated model’s “0.7” means it happens seven times in ten. `class_weight="balanced"` deliberately distorts the intercept to trade precision for recall, so the raw output isn’t a probability at all. Since the number is shown to a clinician next to a threshold, it has to mean what it says. I fit an isotonic calibrator on a held-out slice that never touched the coefficients; expected calibration error went from 0.100 to 0.010. → “Isotonic or Platt?” → *Isotonic: non-parametric, and I had no reason to assume the miscalibration was sigmoidal. It needs more data and can overfit a small calibration set — 1,125 points is enough here.*

17. How do you choose an operating threshold? Not 0.5. It’s the lowest threshold whose precision still clears 0.60, because the errors are asymmetric in **both** directions. A missed serious presentation hurts a patient today. But a prompt that fires too often trains clinicians to dismiss the panel — and then they dismiss the one that mattered. Alarm fatigue is a patient-safety problem, so I gate the false-alarm rate too. → “With real data?” → *From the actual cost ratio: what does a missed escalation cost versus a redundant review, and pick the threshold minimising expected cost.*

18. Walk me through a security issue you found. PostgREST filter injection in patient search. The query was interpolated into `or=(name.ilike.*{q}*,...)`. PostgREST filters are a text grammar, not bound parameters, so a `)` or `,` closed the group early and the rest was parsed as further filter clauses. RLS contained the blast radius to the caller’s own clinic, but the query the database ran wasn’t the one I wrote — which is the definition of injection. The fix quotes values per PostgREST’s escaping rules and neutralises LIKE wildcards, because a bare `%` matched every patient in the clinic. There’s a test with a real parser asserting the payload stays inside one quoted value. *Explain why it’s injection even though RLS contained it. That’s the maturity.*

19. How do you keep PHI out of your logs? Request and response bodies are never logged — transcripts and notes are all PHI, and a log aggregator isn’t a clinical record system. Query parameters are redacted against a deliberately broad list including `q`, because **a log full of `q=Sharma` is a log full of patient names**. Metric route labels collapse ids so no patient gets their own metric series. Audit entries record which *fields* changed, not their values.

20. What happens when every AI provider is down? The degradation ladder (diagram 3). Each rung is a test. → “Why does the risk endpoint keep working?” → *It’s a dot product and a table lookup in-process. A direct benefit of exporting the model to JSON instead of calling a model-serving endpoint.*

Tier 3 — asked by the most senior person in the room

21. Is there RAG in this system? No vector store and no embedding retrieval, and that was a decision rather than an omission. There are two retrieval problems here. Patient history is a bounded set of rows for one patient, ordered by time — SQL is the right tool, and the only one that respects RLS. Clinical knowledge is owned by the external decision-support service. A vector database would have been a keyword on a CV. → *“When would you add one?” → When there’s a corpus too large to scan and too unstructured to index — clinical guideline PDFs, say. Then I’d need chunking, a retrieval evaluation with precision@k, and grounding checks. I wouldn’t add the infrastructure before the corpus. This answer is stronger than pretending to have RAG. Do not pretend.*

22. What’s in this system that you didn’t build? The differential diagnosis, investigations and treatment recommendations come from an external Clinical Synthesis API. I built the integration: request shaping, response normalisation, failure containment, and keeping its secret base URL out of the browser. Claiming the clinical brain would be the single most damaging thing I could say, because one follow-up question exposes it.

23. How would this scale to 500 clinics? RLS scales fine — it’s an indexed predicate, and the clinic-scoped indexes are there. What doesn’t: rate limiting and the spend cap are per-process, so N workers means $N \times$ the limit; both need Redis. Metrics are per-process and need per-instance scraping. And `/consultations` caps at 500 rows with no pagination, which would need cursor pagination. → *“Would you shard per clinic?” → Not at 500. Isolation is already at row level and the data volume is small; sharding would add operational complexity for no benefit.*

24. What would you need before this touched a real patient? Clinical validation on a real labelled cohort with ethics approval. A prospective **silent-mode** evaluation where it runs but influences nothing, so its real-world error rate is measured before anyone acts on it. A DPDP compliance review and a data-protection impact assessment. Penetration testing. A deployed, monitored, backed-up environment with tested restores. And an incident process for when it gets something wrong, because it will. *“Silent mode first” is the phrase that shows you understand clinical rollout.*

25. What did a code reviewer catch that you’d missed? Ten things, and I fixed all of them. The two worth describing: a doubled escape in a regex meant `\\s+` matched a literal backslash followed by “s” rather than whitespace, so any symptom phrase straddling a newline in the model’s prose was missed — which silently disabled the deterministic red-flag criteria that depend on those features. And the citation verifier was blocking the event loop for ten seconds on a long transcript (story 5). Both were invisible: nothing errored, nothing was slow in a way you’d notice locally. *Being able to answer this at all — and having fixed them with tests — is worth more than never having had findings.*

The traps

Never say these

Do not say	Say instead
"It diagnoses patients"	"It surfaces considerations a physician evaluates"
"HIPAA / GDPR / DPDP compliant"	"Designed with DPDP Act 2023 principles in mind; not certified, not audited"
"Clinically validated"	"Evaluated on synthetic data; no clinical validation"
"Tamper-proof audit log"	"Append-only and tamper-evident — a rewrite is detectable"
"Production-ready"	"Working prototype, not deployed"
"I built the clinical decision engine"	"I built the integration; the engine is external"
"99% accurate"	Quote precision, recall, and the data they were measured on
"It's fully tested"	"426 tests; here's what they cover and here's the gap"

Volunteer these before you're asked

- The risk model is trained on synthetic data and is not clinically validated.
- The decision-support engine is external.
- It isn't deployed.
- Rate limiting is per-process.
- There's no fairness evaluation, because the synthetic cohort has no demographic attributes to measure across.
- There's no end-to-end browser test.

Volunteering a limitation costs you nothing and buys you credibility for everything else you say. Being caught hiding one costs you the interview.

If you don't know

Say so, then say how you'd find out. "I don't know off the top of my head — I'd check `eval/results/risk_model.json`, which is committed precisely so I don't have to remember it" is a *good* answer.

Never guess a number.

The night before

Re-read, in this order: the numbers table (page 2) · the 60-second pitch · the five stories · the traps.

Have open in a tab:

- `docs/MODEL_CARD.md` — every ML number, with its caveats
- `backend/eval/results/` — the three JSON files behind every claim
- `supabase/migrations/20260911120003_transactional_finalize.sql` — the function you'll be asked to walk through

Be able to run, live, in under a minute:

```
make eval      # both harnesses, with their gates
make test      # 388 backend tests
```

Being able to *show* the numbers regenerating is worth more than quoting them.

Questions to ask them

Ask two or three. These signal that you think about the same things they do.

1. “How do you handle the gap between a model’s offline metrics and its behaviour once people are acting on it?”
2. “Where do your correctness guarantees live — application code, the database, or types? I put mine in Postgres and I’m curious how that trade-off plays out at your scale.”
3. “What’s the failure that would worry you most in this system, and how would you know it had happened?”
4. “How much of your evaluation runs in CI versus by hand?”

The last thing

You built a system where every claim has a test or a number behind it, you audited your own project and found that several documented behaviours didn’t exist, and you removed the claims rather than quietly leaving them. That is the story. Tell it.